Visión por Computadora 3D

Deep Learning en datos 3D

MSc. Fernando Silva



Limitación CNN 2D

Clase 2

- en la clase 2 vimos que una cámara pierde Z al dividir por la profundidad
- recuperamos Z con estéreo, con sensores o con una red (Depth Anything)
- con Z y K, cada píxel (n, m) vuelve a ser un punto 3D:

$$X = \frac{(n - c_x)\, Z}{a}, \qquad Y = \frac{(m - c_y)\, Z}{b}$$

es la ecuación de la clase 2 despejada al revés, un mapa de profundidad de 640 × 480 se convierte en una nube de puntos

- ya sabemos obtener los puntos... ahora cómo hace una red para entender qué es cada punto?

Tareas igual que en 2D

Tarea	Entrada	Salida	Ejemplo
Clasificación	una nube de N puntos	una etiqueta para toda la nube	es una silla o una mesa?
Segmentación	una nube de N puntos	una etiqueta por punto (N etiquetas)	qué puntos son piso, pared, persona
Detección	una escena completa	cajas 3D con clase	dónde están los autos en un escaneo LiDAR

Representación

	Imagen	Nube de puntos
Estructura	grilla regular de H × W	conjunto de N puntos sueltos
Vecinos	fijos: arriba, abajo, izquierda, derecha	hay que buscarlos
Orden	el píxel (10, 20) siempre es el mismo	el punto 7 puede ser cualquiera
Densidad	uniforme	densa cerca del sensor, rala lejos
Vacío	no existe, todo píxel tiene color	casi todo el espacio está vacío

la convolución necesita una grilla para deslizar el filtro, la nube de puntos no tiene grilla

Propiedades de una red 3D

1. Invariante a permutaciones: N puntos se pueden guardar en N! órdenes distintos y todos son la misma forma. Con N = 1024, N! tiene más de 2600 dígitos.

2. Invariante a transformaciones rígidas: una silla rotada o trasladada sigue siendo una silla.

3. Sensible a vecinos: la forma local (una esquina, un borde, una superficie plana) depende de cómo se ubican los puntos cercanos.

cada arquitectura de la clase se puede leer como una respuesta distinta a estas tres propiedades

Datasets

Dataset	Tipo	Tarea típica
ModelNet40	12 311 modelos CAD, 40 categorías	clasificación
ShapeNet Part	objetos CAD con partes etiquetadas	segmentación de partes
ScanNet / S3DIS	interiores escaneados con RGB-D	segmentación semántica
KITTI / nuScenes / Waymo	LiDAR de autos en la calle	detección 3D
TotalSegmentator	tomografías de cuerpo completo	segmentación médica

Si el problema es que no hay grilla… por qué no la construimos nosotros?

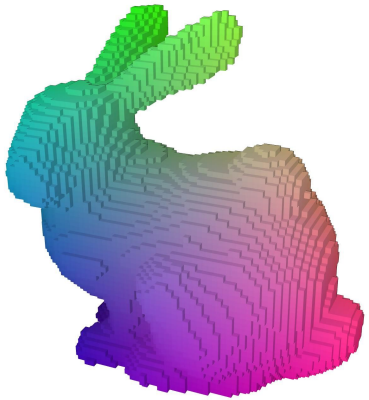
Vóxeles y CNN 3D

Voxelizar

Forzar la grilla

- se divide el espacio en cubos de lado s (por ejemplo, 5 cm)
- cada punto cae en el cubo de índice:

$$(i, j, k) = \left(\left\lfloor \frac{X - X_{min}}{s} \right\rfloor, \left\lfloor \frac{Y - Y_{min}}{s} \right\rfloor, \left\lfloor \frac{Z - Z_{min}}{s} \right\rfloor \right)$$



- cada vóxel guarda ocupación (0 o 1), o el promedio de los puntos que caen dentro
- es lo mismo que pasar de metros a píxeles en la clase 2, pero en tres ejes

CNN 3D

Costo en memoria

- un tensor float32 ocupa 4 bytes por valor, con un solo canal:

Resolución	Vóxeles	Memoria (1 canal)	Memoria (64 canales)
32^3	32.768	128 KB	8 MB
128^3	2.097.152	8 MB	512 MB
512^3	134.217.728	512 MB	32 GB

- duplicar la resolución multiplica la memoria por 8: crece con el cubo, no con el cuadrado

CNN 3D

Ejemplo

- un escaneo LiDAR de una calle, de 70 m × 80 m × 4 m, con vóxeles de 5 cm:
  - grilla: 1400 × 1600 × 80 = 179 200 000 vóxeles
  - un escaneo típico trae del orden de 120 000 puntos
  - aunque cada punto caiga en un vóxel distinto, se ocupa menos del 0.07 % de la grilla
- una CNN 3D densa gasta más del 99.9 % del cómputo multiplicando ceros
- los puntos viven sobre superficies (2D) dentro de un volumen (3D), por eso casi todo está vacío

Vóxeles y CNN 3D



LINK

Vóxeles y CNN 3D

1. un ejemplo con una grilla pequeña de R = 4, es decir 4×4×4 = 64 cubos, y un objeto de 3 puntos:

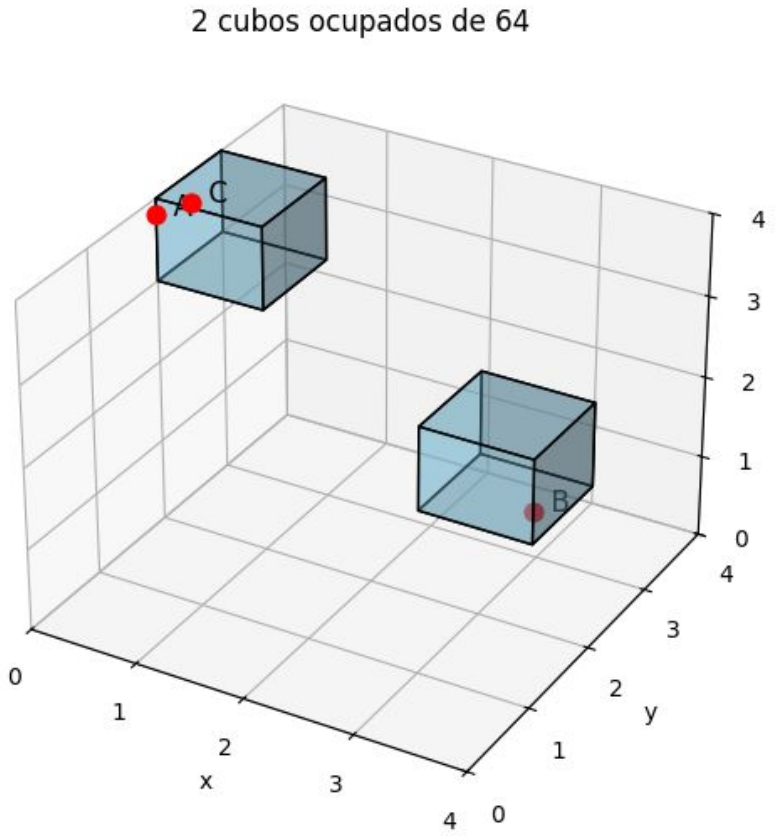
- A = (-1.0, 0.0, 0.9)
- B = (1.0, -0.5, 0.2)
- C = (-0.9, 0.1, 0.95)

Punto	(P+1)/2 … [0,1]	× 4 … [0,4]	.long()	.clamp(0,3)
A	(0, 0.5, 0.95)	(0, 2, 3.8)	(0, 2, 3)	(0, 2, 3)
B	(1, 0.25, 0.6)	(4, 1, 2.4)	(4, 1, 2)	(3, 1, 2)
C	(0.05, 0.55, 0.975)	(0.2, 2.2, 3.9)	(0, 2, 3)	(0, 2, 3)

2. creamos la grilla vacía, V tiene forma (1, 1, 4, 4, 4): 1 objeto, 1 canal y 64 cubos, todos en 0

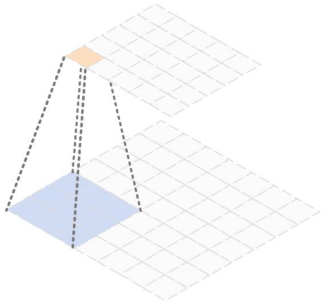
3. marcar los cubos ocupados
   - V[0, 0, 0, 2, 3] = 1 # A y C
   - V[0, 0, 3, 1, 2] = 1 # B

2 cubos en 1 y 62 en 0, como A y C caen en el mismo cubo, cuentan como uno solo, y así se pierde el detalle dentro de cada cubo.

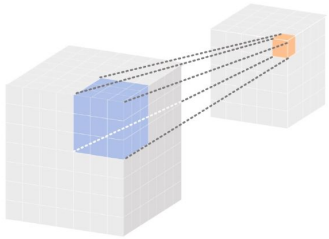


Vóxeles y CNN 3D

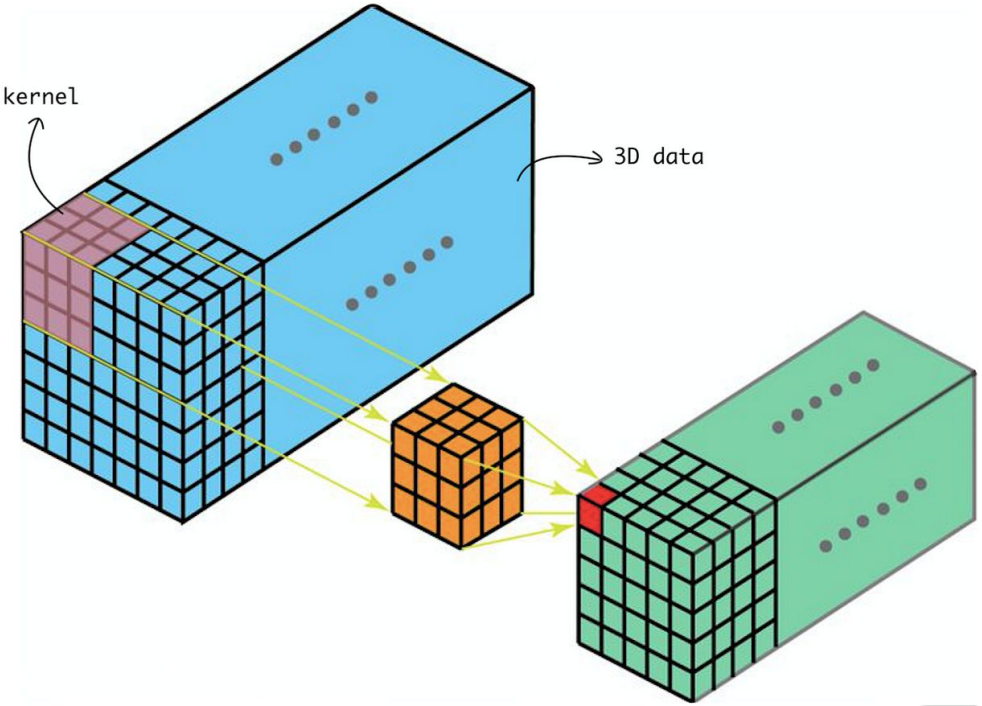
Conv3D:



2D-Conv
Kernel size:3*3
Single Channel



3D-Conv
Kernel size:3*3*3
Single Channel



Ejercicio, Vóxeles y CNN 3D



LINK

Convoluciones Dispersas (Sparse)

Limitación: voxelizar siempre pierde detalle dentro de cada cubo, y la resolución obliga a elegir entre precisión y memoria, las convoluciones dispersas resuelven la memoria, pero no el detalle perdido

voxelizar es forzar la nube a parecerse a una imagen… y si en lugar de cambiar los datos cambiamos la red?

PointNet, PointNet++ y DGCNN

PointNet (2017)

Si el orden de los puntos no importa, la red tiene que usar una función que tampoco dependa del orden: una función simétrica, como la suma o el máximo.

Abstract

Point cloud is an important type of geometric data structure. Due to its irregular format, most researchers transform such data to regular 3D voxel grids or collections of images. This, however, renders data unnecessarily voluminous and causes issues. In this paper, we design a novel type of neural network that directly consumes point clouds, which well respects the permutation invariance of points in the input. Our network, named PointNet, provides a unified architecture for applications ranging from object classification, part segmentation, to scene semantic parsing. Though simple, PointNet is highly efficient and effective. Empirically, it shows strong performance on par or even better than state of the art. Theoretically, we provide analysis towards understanding of what the network has learnt and why the network is robust with respect to input perturbation and corruption.

la misma MLP aplicada a cada punto por separado (pesos compartidos)

toma, en cada dimensión, el valor más alto entre todos los puntos

$$f(\{P_1, \ldots, P_N\}) \approx \gamma \left(\max_{i=1..N} h(P_i) \right)$$

otra MLP que decide la clase a partir de ese resumen

los autores demuestran que esta forma puede aproximar cualquier función continua sobre conjuntos de puntos. El max es la pieza que da la invarianza.

PointNet (2017)

Puntos críticos

- el max se queda, en cada dimensión, con un solo punto ganador
- los puntos que ganan al menos una dimensión forman el conjunto crítico, a lo sumo 1024 puntos
- se pueden borrar todos los demás y la salida no cambia
- el conjunto crítico suele parecerse al esqueleto del objeto, por eso PointNet es robusto a puntos faltantes, esto se ve en la práctica

Limitación: cada punto se procesa solo y la única mezcla entre puntos es el max global. PointNet nunca mira a los vecinos, así que le cuesta captar detalles locales como un borde o una esquina

cómo le devolvemos la vista local sin perder la invarianza?

PointNet++ (2017)

La idea es aplicar PointNet por regiones, de forma jerárquica, igual que una CNN pasa de bordes a partes y de partes a objetos.

Cada nivel (set abstraction) tiene tres pasos:

1. **muestreo:** se eligen M centros con farthest point sampling, que escoge cada nuevo punto lo más lejos posible de los ya elegidos

2. **agrupamiento:** para cada centro se toman los vecinos dentro de una esfera de radio r (ball query)

3. **mini PointNet:** cada grupo se resume en un vector

- de N puntos se pasa a M < N puntos, cada uno con un feature que describe su región
- para segmentar se vuelve a la resolución original interpolando con los 3 vecinos más cercanos, pesados por la inversa de la distancia

farthest point sampling cubre la forma de manera pareja; un muestreo al azar dejaría huecos donde hay pocos puntos.

DGCNN (2019)

se arma un grafo conectando cada punto con sus k vecinos más cercanos (kNN)

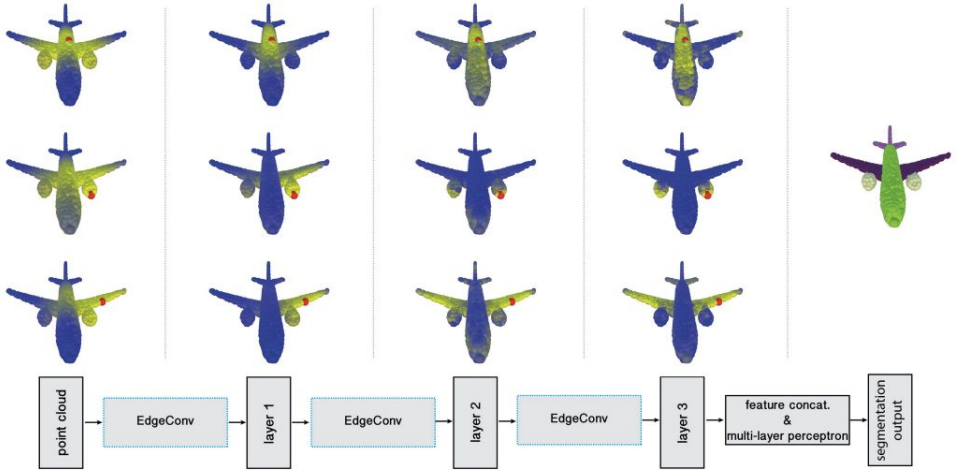


Fig. 1. **Point cloud segmentation using the proposed neural network.** Bottom: schematic neural network architecture. Top: Structure of the feature spaces produced at different layers of the network, visualized as the distance from the red point to all the rest of the points (shown left-to-right are the input and layers 1-3; rightmost figure shows the resulting segmentation). Observe how the feature space structure in deeper layers captures semantically similar structures such as wings, fuselage, or turbines, despite a large distance between them in the original input space.

EdgeConv calcula un feature por arista y luego toma el max:

dice dónde está el punto

$$x'_i = \max_{j \in \mathcal{N}(i)} h_\theta\left(x_i,\ x_j - x_i\right)$$

dice cómo se ve su entorno

el grafo se recalcula en cada capa usando los features, no las coordenadas: dos patas de una silla pueden quedar como vecinas aunque estén lejos en el espacio

DGCNN (2019)

Resultados

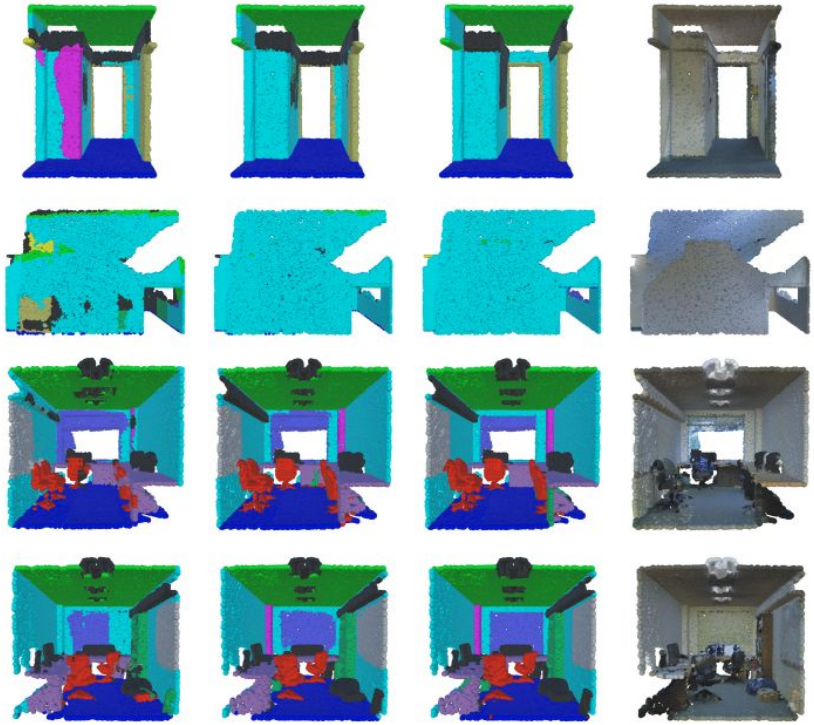


Fig. 10. Semantic segmentation results. From left to right: PointNet, ours, ground truth and point cloud with original color. Notice our model outputs smoother segmentation results, for example, wall (cyan) in top two rows, chairs (red) and columns (magenta) in bottom two rows.

PointNet, PointNet++ y DGCNN



LINK